CS 417/545 Project 2 Background: Primary-Backup Replication

Read this before you start writing code if you have not seen primary-backup replication before. It gives you the vocabulary and the mental model you will need for the Project 2 writeup and lecture L9b. Fifteen minutes.

0. The ML connection: parameter servers

In Project 1 we built fault-tolerant distributed-training workers. But a real training system has TWO fault-tolerance problems:

1. Workers can die. (P1 solved this.)
2. The shared parameter store can die. (This is P2.)

In a parameter-server architecture every worker runs the same loop:

```
loop:
    weights  = GetWeights(ps)
    gradient = compute_grad(weights, local_minibatch)
    PutWeights(ps, weights - lr * gradient)
```

If the parameter server crashes, every worker is stuck. The fix is to replicate the parameter server. That is literally what P2 asks you to build: a key-value store with a primary and a backup, where keys are parameter names and values are parameter values.

Real systems that do this:

- **ps-lite / Mu Li et al.**, "Parameter Server for Distributed Machine Learning," OSDI 2014.
- **TensorFlow (early distributed mode).** `ps` and `worker` jobs, replicated via Chubby internally.
- **PyTorch** `torch.distributed.rpc`**.** Supports a replicated PS pattern.
- **Horovod + Elastic.** Different flavour (all-reduce) but same motivation.

For P2 the "model" is small (10 weights) and the "SGD step" is one `Get` + one `Put` on a single key, but the structure is the same as in the real systems.

1. The problem we are solving

A single key-value server is a single point of failure. If it crashes, every client is stuck. We want a service that keeps its state SAFE across machine failures.

The cheapest answer is to run TWO servers that hold the same state. When the first one dies, the state still exists on the second. That is primary-backup replication.

In P2 we build the replication protocol itself. We do not yet build the failover machinery that detects the primary is down and redirects clients to the backup. That is scope for the next project (`p3-future-scope.md`).

2. State-machine replication

The idea behind every replication protocol in this course:

- Treat the service as a state machine.
- Agree on the order of operations.
- Apply the same operations in the same order on every replica.
- Every replica ends up in the same state.

Primary-backup is one way to agree on the order: whatever the primary decides to apply, in whatever order it decided, is the order the backup must apply. Paxos (L10) is a different way.

For primary-backup to be correct, two things must hold on every mutating operation:

1. The primary applies the operation to its OWN state.
2. The primary waits for the backup to confirm it also applied the operation BEFORE replying to the client.

If the primary skips step 2 and crashes right after the reply, the backup never saw that operation. A client that reads from a new primary later would see a value that "went backwards" in time. Linearisability broken.

3. At-most-once, revisited

In P1 we discussed and implemented the at-most-once pattern: every operation carries an Xid, and the server keeps a table of Xids it has already applied so a retry does not apply the operation twice.

P2 has the same situation on both sides. The primary can see the same client operation twice (original attempt plus a retry after a timeout). The backup can see the same forwarded operation twice (primary retried its ForwardOp after the first ack was lost). Both servers must deduplicate against the Xid, and both must return the same reply they returned the first time they saw it.

Use the same table pattern from P1 Section 3.3: a map from Xid to the reply produced. Keep the two tables consistent; they are the single source of truth for whether an operation has been applied.

4. Where consensus comes in

Primary-backup, as implemented in P2, gets us as far as "two machines hold the same state." It does not yet handle the primary actually crashing: there is no automatic promotion of the backup, no way for the clerk to notice and reconnect, no mechanism to stop a partitioned-off primary from continuing to serve stale writes.

Those gaps are exactly what consensus solves. L9b walks through the specific corner cases: ghost pings, clock skew between the view server and the primary, and the problem of replicating the view server itself with no higher authority to break its ties. L10 (Paxos) is the way out. P3 (next project) builds on it.

5. Suggested reading

- Lamport, "The Part-Time Parliament" (1998), the original Paxos paper. Skim; read "Paxos Made Simple" (2001) instead if you want the readable version.
- Chandra, Griesemer, Redstone, "Paxos Made Live" (2007). How Chubby, the real Google lock service, actually implements this.
- van Renesse, Schneider, "Chain Replication" (OSDI 2004). A different primary-backup variant used by Azure Storage.

None of these are required for P2. They are good background for L10 and L11.